



Project Report – Justice Flow AI Lab Project

Name:

**Muhammad Furqan Sheikh
Muhammad Abu Bakar
Abdullah Muaz Abbasi**

NU ID:

24K-0527, 24K-0826, 24K-0764

April 28th, 2026

—

Syllabus:

Artificial Intelligence Lab

Syllabus Code: AL-2002

—

Instructor:

Sir Atif Luqman

Project Team & Subsystem Allocation:

Muhammad Abu Bakar – Subsystem 1 Engine (Crime Hotspot Analyzer)
Abdullah Muaz Abbasi – Subsystem 2 Engine (Case Priority Recommender)
Muhammad Furqan Sheikh – Subsystem 3 Engine (Automated Workload Balancer)

1. Executive Summary

The JusticeFlow AI tier demonstrates a highly mature, decoupled, and distributed architecture. Rather than relying on a centralized, monolithic AI layer, the intelligence capabilities are distributed across three distinct subsystems. Each subsystem employs an independent Python daemon process tailored to its specific domain, interacting asynchronously with a PostgreSQL backend. The implementation showcases a robust understanding of data science concepts (unsupervised clustering, supervised ensembles) and applied mathematics (combinatorial optimization, dynamic cost matrix generation) applied across different operational boundaries.

2. Technical Evaluation by Subsystem

2.1 Subsystem

1: Crime Hotspot Analyzer (Unsupervised Learning)

Algorithm: DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Lead Developer: Muhammad Abu Bakar

Technical Assessment:

Data Sanitization:

The module implements strict geospatial boundary checking, actively filtering out invalid latitude/longitude data before clustering, which prevents algorithmic crashing or skewed centroid calculations.

Algorithmic Precision:

The use of the Haversine metric via the ball_tree algorithm correctly accounts for the curvature of the Earth when calculating distances between incident coordinates.

Thresholding: The parameters (1.5km radius, minimum 5 incidents) represent a realistic policing heuristic. The system successfully isolates "noise" (isolated incidents) to ensure patrol resources are only recommended for statistically dense areas.

Database Integration:

Seamlessly outputs the top 5 high-risk zones into the analytics.hotspots schema with automated timestamping and rank ordering.

2.2 Subsystem

2: Case Priority Recommender (Supervised Learning)

Algorithm: Random Forest Classifier

Lead Developer: Abdullah Muaz Abbasi

Technical Assessment:

Model Architecture:

Utilizes a highly stable ensemble method ($n_estimators=150$, $max_depth=6$) to prevent overfitting on limited historical crime data.

Evaluation Pipeline:

Implements a strict train/test split (80/20) with stratification, ensuring the model's accuracy and classification reports are mathematically valid before executing inference on open cases.

Probability Ranking:

The system utilizes `predict_proba()` rather than simple binary classification. This allows the subsystem to sort open cases into a ranked queue based on their exact probability of requiring high priority escalation, ensuring the most urgent cases are handled first.

2.3 Subsystem

3: Automated Workload Balancer (Combinatorial Optimization)

Algorithm: Hungarian Algorithm (Linear Sum Assignment)

Lead Developer: Muhammad Furqan Sheikh

Technical Assessment:

Heuristic Cost Modeling:

Constructs an advanced $N \times M$ cost matrix that dynamically calculates assignment penalties based on four rigorous metrics: Workload Ratio (35%), Skill Match (30%), Rank Suitability (20%), and Geographic Haversine Distance (15%).

Optimal Assignment:

By utilizing SciPy's `linear_sum_assignment`, the module guarantees a mathematically optimal distribution of cases that minimizes systemic strain across the precinct, strictly respecting the maximum threshold of 10 cases per officer.

Data Persistence:

Generates detailed recommendation logs, including cost breakdowns and penalty scores, inserting them cleanly into the `analytics.Officer_Workload_Assignments` table for scheduling and review.

3. Architectural & Engineering Assessment

Fault Tolerance:

All three modules employ extensive try/catch/finally blocks and automated database cursor closure, ensuring that database timeouts or calculation errors in one subsystem do not cascade into or crash the others.

Stateless Execution: The models are designed to wake up, execute their specific queries, write the output, and terminate. This stateless daemon approach is highly resource-efficient and prevents memory bloat over long uptimes across the subsystems.

Schema Segregation: The decision to isolate AI outputs into a dedicated analytics schema (rather than mutating core transactional tables) protects the integrity of the primary database and represents excellent cross-subsystem database design.